NetOpsAgent: A Hierarchical Multi-Agent LLM Framework for Autonomous Cloud Infrastructure Incident Management

Kinjal Vaishnav
Oracle Cloud Infrastructure, Seattle, WA, USA
kinjal.vaishnav@oracle.com

***Abstract*—Cloud infrastructure incidents cause significant service disruption, yet mean time to root cause remains long due to alert volume and telemetry complexity. We present NetOpsAgent, a hierarchical multi-agent LLM framework for autonomous cloud incident management, decomposing investigation into four agents—Supervisor, Telemetry, Reasoning, Action—coordinated via a shared `InvestigationState` object, with a confidence-gated tiered autonomy mechanism bounding autonomous-action risk. We evaluate on two independent real fault-injection benchmarks: AIOps Challenge 2020 (N=75) and RCAEval RE1 (N=75). Across six pre-registered configurations—including a raw-context fix, RAG with and without relevance gating, and a test designed specifically to isolate decomposition's benefit—we find no statistically significant multi-agent advantage over a single-agent baseline on either dataset (McNemar $p > 0.05$ throughout), and a rule-based baseline stays competitive with both. We then test a targeted extension: a Falsifier agent adversarially validating the Reasoning agent's hypothesis against evidence it did not use, mirroring the validation mechanism recent RCA multi-agent systems report gains from. This significantly improves accuracy on single-entity incidents (33.3% vs. 21.3%, $p = 0.023$) but significantly regresses multi-service incidents (24.0% vs. 42.7%, $p < 0.001$): the extension lacks service-dependency-graph awareness and mistakes a downstream symptom for the root cause once more than one service is involved. Adding real service-topology data eliminates the regression ($p = 0.50$, was $p < 0.001$) but does not yet produce a gain there. A control test shows the AIOps gain comes from the Falsifier capability itself, not the decomposition it was attached to: the same Falsifier on a single-agent hypothesis scores statistically indistinguishably from the multi-agent version ($p = 1.0$ head-to-head). We report this in full, characterize why self-reported LLM confidence fails to support safe autonomous action here, and identify the likely mechanism (blind metric selection under decomposition) behind the base architecture's underperformance. A reference implementation is publicly available under the MIT license.**

***Index Terms*—multi-agent LLM systems, AIOps incident management, autonomous root cause analysis, site reliability engineering, retrieval-augmented generation**

I. Introduction

Site reliability engineering in large-scale cloud environments faces a structural tension: incident frequency scales with system complexity, but human investigation capacity does not, and mean time to root cause for complex distributed failures commonly exceeds 30–60 minutes [1], [14]. LLMs offer a qualitatively different approach: instruction following, tool use, and multi-step reasoning over unstructured text make them natural candidates for investigation tasks historically requiring senior engineers. Yet naive single-agent application fails in practice—the context window fills with raw telemetry before the model can reason causally, with no principled mechanism for deciding when confidence is sufficient to act autonomously.

We present NetOpsAgent, a hierarchical multi-agent LLM framework addressing both limitations, decomposing investigation into four specialized agents—Supervisor, Telemetry Investigation, Reasoning, Action—each handling a distinct phase of the response workflow. A shared `InvestigationState` object propagates structured findings between agents, enforcing context hygiene without information loss, and a confidence-gated tiered autonomy mechanism lets the system act autonomously on high-confidence hypotheses while escalating uncertain cases to humans.

The framework targets two observed single-agent failure modes. *Context pollution*: given access to all observability tools, a single agent accumulates raw metrics, logs, and configuration data in one context window, too large and heterogeneous for reliable causal reasoning, particularly where the causal signal is weak relative to background noise. *Uncalibrated autonomy*: without principled confidence scoring, an LLM agent takes either too many autonomous actions (increasing blast radius) or too few (eliminating the productivity benefit). NetOpsAgent is designed to address both; Section V reports how well that design holds up under controlled measurement.

This paper makes four contributions: (1) the NetOpsAgent architecture and its design rationale; (2) a controlled empirical evaluation across two independent real datasets and six architectural configurations, showing that multi-agent decomposition and retrieval augmentation do not produce a significant improvement over a single-agent baseline for this task—including under a test designed specifically to favor the multi-agent hypothesis; (3) a failure-mode and confidence-calibration analysis, including a real finding that self-reported and self-consistency LLM confidence both fail to discriminate correct from incorrect hypotheses in this setting; and (4) a targeted extension experiment adding a Falsifier/validation agent, showing a statistically significant improvement on single-entity incidents and a statistically significant regression on multi-service incidents, with the divergence traced to a specific, addressable mechanism (missing service-dependency-graph awareness) rather than left as an unexplained split.

II. Related Work

A. LLM Agents and Tool Use

ReAct [2] established the observe-reason-act loop as the foundational pattern for LLM agents with external tool access. Subsequent work—Toolformer [3], function-calling APIs such as Gorilla [9]—showed LLMs reliably invoke structured tools given well-defined schemas. LangGraph [4] introduced stateful, cyclical agent graphs with persistent state across tool invocations; NetOpsAgent builds directly on this, using LangGraph's `StateGraph` as the orchestration primitive.

B. AIOps, Benchmarks, and Multi-Agent Coordination

Classical network anomaly detection relies on threshold alerting, statistical process control, and ML classifiers on labeled KPI time series [5], consistent with standardized observability data formats [32]; the wider AIOps landscape spans log parsing [17] to runbook execution [18], typically rule-based or ML pipelines with limited cross-incident-type generalization. NetOpsAgent consumes the alert these systems produce as input and addresses the investigation phase that follows. SWE-bench [10], [19] evaluates agents on asynchronous software tasks; our evaluation (Section V) instead draws ground-truth labels directly from AIOps Challenge 2020 [5] and RCAEval, rather than constructing author-labeled traces. On LLM-based RCA: Guo et al. [23] apply GPT-4 to log parsing with strong zero-shot but limited causal reasoning; Chen et al. [24] propose trace-based LLM RCA for microservices; Luo et al. [25] use a knowledge-graph approach; Roy et al. [26] describe single-agent RAG-based triage. NetOpsAgent differs in hierarchical decomposition, confidence-gated tiered autonomy, and an ablation measuring decomposition's own contribution (Section V). LLM-based multi-agent coordination itself differs from classical MAS research in using natural language and JSON rather than formal ontologies: AutoGen [7] and MetaGPT [8] show conversational patterns solve software engineering tasks in low-risk settings, while NetOpsAgent targets reliability-critical contexts—typed state propagation, confidence-gated action safety, full audit logging; Section V evaluates how much the first of these actually contributes on real data.

C. Positioning Relative to Recent RCA Multi-Agent Systems

Two recent systems decompose RCA into specialized LLM agents and ground hypotheses in retrieved external knowledge, much as NetOpsAgent does. Flow-of-Action [34] pairs a main investigation agent with auxiliary agents that retrieve, generate, and execute SOP steps, reporting 64.01% root-cause accuracy vs. a 35.50% ReAct baseline on its own industrial benchmark. MA-RCA [35] separates a Retrieval Agent (RAG grounding) from a Validation Agent (dynamic-testing verification), reporting 95.2% F1 on Nezha and 82.8% F1 on a power-metering dataset.

NetOpsAgent's four-agent decomposition—Supervisor, Telemetry Investigation, Reasoning, Action, with RAG inside the Reasoning agent—is an instance of the same general pattern: specialize by cognitive function, ground reasoning in retrieved precedent, keep verification structurally separate from generation where possible. We do not claim architectural novelty over Flow-of-Action or MA-RCA, and have narrowed this paper's contribution accordingly.

This paper instead contributes a controlled empirical comparison, under matched task, telemetry, and LLM backbone conditions, of whether that general pattern outperforms a single well-prompted agent on two independent real datasets. Section V finds decomposition alone does not, unlike the substantial gains Flow-of-Action and MA-RCA report on their own benchmarks. We go further and test MA-RCA's specific mechanism—a dedicated validation stage—directly against our data (Section V-G): it helps on single-entity incidents and, without service-topology awareness, hurts on multi-service ones—a falsifiable account of *when* this pattern pays off, not a blanket negative or unqualified replication.

This null result is consistent with concurrent findings outside RCA: Xu et al. [36] show *homogeneous* multi-agent workflows (agents sharing one base model) are matched by a single well-tuned agent and argue a real advantage needs *heterogeneous* models—NetOpsAgent already satisfies that (GPT-4o for Reasoning, GPT-4o-mini for Telemetry, Section IV) and still shows no gain; Su and Wu [37] explain why theoretically—decomposition helps for *independent* subtasks and loses its advantage as they become interdependent, closer to whole-incident diagnosis, consistent with the blind-metric-selection mechanism in Section VI.A.

D. Confidence Estimation in LLM Systems

Verbalized confidence elicitation—asking the model for a numeric score in the spirit of chain-of-thought prompting [6]—correlates with accuracy on some factual tasks, though calibration degrades out-of-distribution [33]. In SRE, an overconfident incorrect hypothesis can trigger autonomous actions that extend an incident. Section V measures this directly: verbalized confidence does not separate correct from incorrect hypotheses here, and self-consistency fails for the same reason—an open problem, not a solved one.

III. Framework Architecture

A. System Overview

NetOpsAgent comprises four agents coordinated via a LangGraph `StateGraph`. All agents share a single `InvestigationState TypedDict` that accumulates findings, tool results, and confidence scores across the investigation lifecycle. The Supervisor Agent acts as the graph entry point and routing controller; the three worker agents—Telemetry Investigation, Reasoning, and Action—are dispatched sequentially or conditionally based on Supervisor routing decisions.

This is a hierarchical, not linear-pipeline, architecture: the Supervisor does not simply hand off to the next stage in a fixed sequence—it classifies each incident, generates the routing plan the Telemetry Agent executes, and retains authority to interrupt or redirect the pipeline mid-investigation (Section III.C). Control flows top-down from Supervisor to worker agents, while data flows bottom-up through

`InvestigationState` back to where routing decisions are made—the defining property of a hierarchical, versus strictly sequential, design.

A central design principle is context hygiene via decomposition: each agent receives only the information relevant to its task, not the full raw telemetry stream. The Telemetry Agent receives the alert trigger and produces structured findings; the Reasoning Agent receives those findings (not raw tool responses) and produces a hypothesis; the Action Agent receives the hypothesis and confidence score. This staged flow is intended to prevent context pollution from degrading reasoning quality, tested directly in Section V. Fig. 1 shows the full architecture.

B. InvestigationState Schema

All agents communicate exclusively via a shared `InvestigationState TypedDict` with key fields `alert_trigger` (External: type, service, severity), `routing_plan` (Supervisor: ordered tool query plan), `findings` (Telemetry: typed signal/value/timestamp list), `rag_context` (Reasoning: top-5 postmortem chunks), `hypothesis` (Reasoning: root cause + mechanism), `confidence` (Reasoning: float [0,1]), `actions_taken` (Action: dispatched action log), and `tool_errors` (Telemetry: schema/timeout failures). This enforces strict information boundaries—no agent reads another's internal context—intended to prevent the context accumulation failure mode seen in single-agent baselines, and provides a natural audit log: the final state records every finding, hypothesis, and action decision.

C. Supervisor Agent

The Supervisor Agent receives the alert trigger, classifies the incident by layer (network, infrastructure, application) and severity, generates a routing plan for the Telemetry Agent, and can interrupt or redirect the pipeline if intermediate findings indicate a change in scope.

D. Telemetry Investigation Agent

The Telemetry Investigation Agent executes iterative tool calls against mocked observability APIs—Prometheus [16] (metrics), ELK [29] (logs), and a CMDB—in a ReAct loop (querying, observing partial results, forming follow-up queries) until it assembles sufficient evidence or exhausts its query budget. Its output is a structured findings object appended to `InvestigationState`: a typed list of (signal, value, timestamp, significance) tuples, not raw API responses—the primary mechanism controlling context size between agents.

E. Reasoning Agent

The Reasoning Agent receives structured findings from the Telemetry Agent and generates a root-cause hypothesis using chain-of-thought prompting [6] and RAG [13], querying a FAISS [12] vector index of historical postmortems for precedent cases similar to the current findings. Output is a hypothesis tuple: (root_cause, mechanism, confidence_score, supporting_evidence, contradicting_evidence). The confidence score (0–1) comes from a structured self-evaluation prompt rating evidence quality, hypothesis specificity, and absence of contradicting signals, and drives the downstream action-safety gate; Section V evaluates how well it actually does so.

F. Action Agent and Confidence Gate

The Action Agent implements a three-tier autonomy model keyed on hypothesis confidence:

- **Tier 1** ($c \geq 0.75$): Low-risk corrective actions (BGP timer adjustments [15], instance restarts, cache flushes) executed autonomously.
- **Tier 2** ($0.6 \leq c < 0.75$): Medium-risk actions (configuration changes, traffic rerouting) queued for human approval with a pre-populated runbook suggestion.
- **Tier 3** ($c < 0.6$): High-risk actions (data migrations, teardowns) never executed autonomously; human operator notified with full investigation context.

All actions, autonomous and queued, are logged to `InvestigationState` with the full hypothesis evidence chain for post-incident audit.

G. Falsifier Extension (Evaluated Separately from the Base Design)

Section V additionally evaluates an optional fifth agent, a Falsifier, inserted between Reasoning and Action. Unlike the four agents above, it is not part of the base architecture evaluated in Sections V.A–F; we describe and evaluate it separately in Section V-G since its contribution depends on incident structure in a way the base components do not. The Falsifier receives the Reasoning agent's hypothesis with the complete raw evidence pool, including signals its summary did not surface, and is prompted to actively look for contradicting evidence rather than confirm the hypothesis, returning a `confirmed`/`contested`/`refuted` verdict; a `refuted` verdict with a concrete, better-supported alternative overrides the hypothesis before Action acts. This mirrors the *role* of MA-RCA's Validation Agent [35], not its mechanism: MA-RCA executes dynamic checks against the system, ours only re-examines existing telemetry via an LLM call—a cheaper, weaker proxy for the same role.

IV. Implementation

A. Technology Stack

The prototype is implemented in Python 3.11: LangGraph 0.1 for orchestration, Google ADK for tools. GPT-4o [27] drives Supervisor and Reasoning; GPT-4o-mini drives Telemetry Investigation. Section V reports measured, not anecdotal, latency and cost. Model assignment follows cost/latency-vs-stakes: Telemetry issues up to 15 tool calls per investigation so cost/latency compound with volume, while Supervisor and Reasoning each make one higher-stakes call—hence GPT-4o-mini for telemetry, GPT-4o for supervision/reasoning. The model-choice ablation supports this: GPT-4o-mini matches

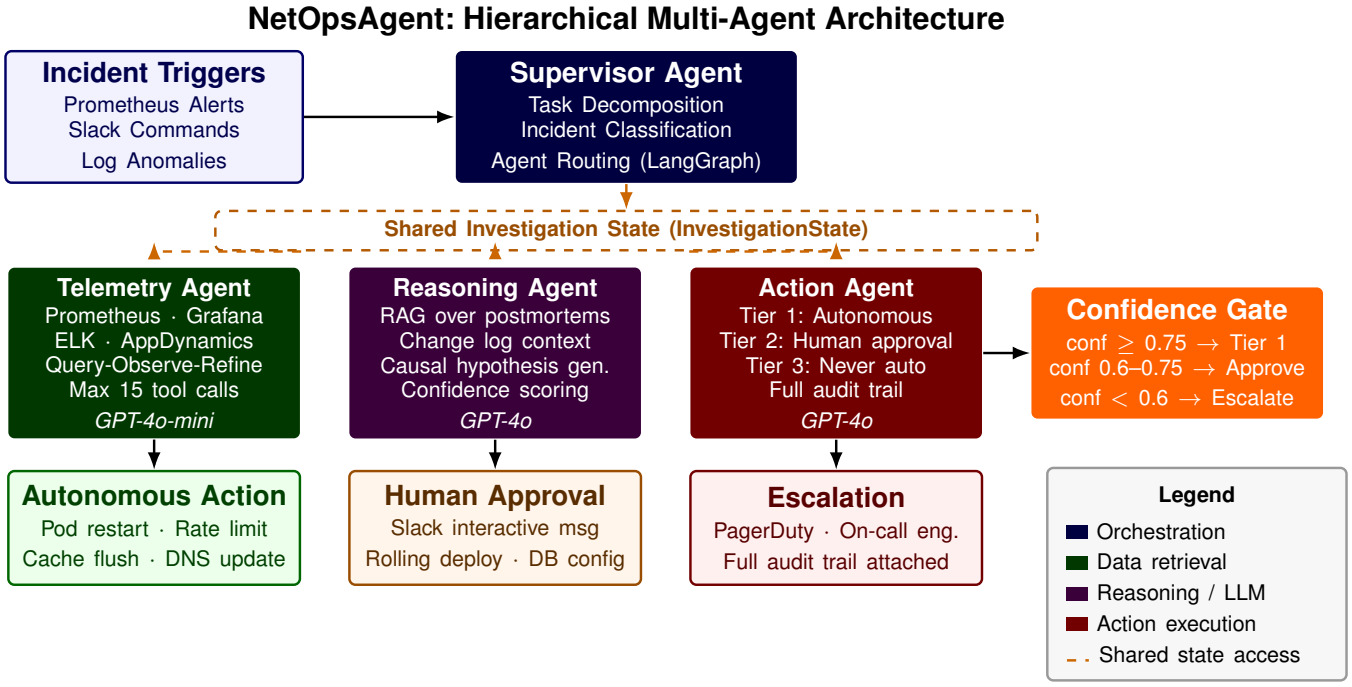


Fig. 1. NetOpsAgent hierarchical multi-agent architecture for autonomous cloud network incident management. The Supervisor Agent routes incoming alerts to the Telemetry Investigation Agent, which executes iterative tool calls against mocked observability APIs. Structured findings propagate to the Reasoning Agent, which applies chain-of-thought reasoning and RAG retrieval to generate a root cause hypothesis with a confidence score. The Action Agent dispatches remediation actions according to the confidence-gated three-tier autonomy model. All agents share a single `InvestigationState` object.

GPT-4o on AIOps, trails on RCAEval, at roughly one-seventeenth the per-token cost.

B. Observability Tool Layer

Observability APIs are mocked via a deterministic trace-replay layer emitting realistic metric/log responses from stored scenarios. Tool failures surface as structured exceptions (e.g., "Prometheus query timed out after 10s; returned data covers 60% of requested window") so the agent reasons about incompleteness rather than crashing.

C. RAG Index Construction

The Reasoning Agent retrieves historical incident context via vector similarity search: postmortems chunked (512 tokens, 64-token overlap), embedded with `text-embedding-3-small` (1,536 dims [11]), stored in a FAISS flat index [12]. The agent retrieves the top-5 most similar chunks by cosine similarity, with recency decay ($\lambda = 0.95$/month) applied before ranking. Section V evaluates a generic, incident-independent version with and without relevance gating.

D. Tool Validation and Error Handling

A three-layer strategy handles errors: tool calls validated against JSON Schema before execution (schema); timeouts capped per class (Prometheus 10s, ELK 15s, CMDB 5s, execution); failures logged to `InvestigationState` and surfaced to the Supervisor for compensating routing (state).

E. Code Availability

The extraction pipeline and evaluation harness producing every Section V result—incident extraction, all six configurations, the Falsifier extension, topology-aware fix, single-agent control, raw per-incident outputs—is available at https://github.com/Kinjal-Oza/multi-agent-observability-demo under the MIT license: the summary-based harness used for evaluation, not a full iterative tool-calling deployment against live APIs.

V. Evaluation

A. Dataset Construction

We evaluate on two independent real datasets, rather than one real dataset plus author-constructed synthetic scenarios as in our original submission. **AIOps Challenge 2020**: 81 ground-truth incidents from the public KPI dataset [5], 75 usable after requiring a valid pre-incident baseline window (6 excluded); labels come directly from published anomaly annotations, no author-assigned labels. **RCAEval RE1** (Zenodo, CC-BY-4.0): 75 independent fault-injection scenarios across three real microservice systems (Online Boutique, Sock Shop, Train Ticket), one run per unique service×fault-type combination of six repeated runs each ships with, avoiding pseudoreplication; ground truth is the dataset's own injected fault label. Both datasets carry externally published ground truth, so this revision no longer needs the author-assigned synthetic labels and inter-rater reliability check (κ) [22] from the original submission's 80-trace synthetic set—a source of potential labeling bias this design removes rather than mitigates.

B. Baselines

Two baselines are evaluated on each dataset: the *single-agent baseline* receives the same alert trigger and tool schema as NetOpsAgent but runs as one GPT-4o agent without decomposition, same task description and completion condition (hypothesis with confidence score)—directly testing decomposition's contribution; the *rule-based baseline* implements a fixed-heuristic classifier on metric-threshold crossing, a pre-LLM production approach.

C. Architectural Configurations Evaluated

Six configurations were run once each on each dataset, pre-registered before execution so the reported outcome could not be selected after the fact:

1) **Baseline** multi-agent (mini-summary → reasoning).
2) **Sort-fix**: rank findings by relative deviation from baseline, not raw magnitude (AIOps only).
3) **Raw-context fix**: the Reasoning Agent sees raw signal data, not only the Telemetry Agent's summary.
4) **+RAG**: real embeddings (`text-embedding-3-small`), a 21-chunk generic operational-knowledge corpus, top-5 retrieval.
5) **+RAG, gated**: as above, restricted to cosine similarity ≥ 0.45 (threshold fixed from a diagnostic sample before this ablation, not tuned to its outcome).
6) **Fair context-pollution test**: single-agent gets a true unranked, unfiltered raw telemetry dump (this paper's definition of context pollution); multi-agent's Telemetry Agent performs real iterative, blind metric-name-based selection first (this paper's definition of context hygiene via decomposition)—the only configuration testing either side against the architecture's own motivating hypothesis.

The generic RAG corpus (Section IV) contains 21 chunks of general SRE knowledge (CPU, memory, disk, network-delay, network-loss, database-layer triage), authored before any evaluation ran, with no incident-specific text—independence from the evaluation traces is true by construction and verifiable by inspection, not merely asserted.

D. Results

Table I reports exact-match accuracy for every configuration on both datasets. McNemar's test [21] on discordant pairs returns $p > 0.05$ for every configuration on both datasets except the fair context-pollution test ($p < 0.001$), significant *favoring the single agent*—opposite the paper's original hypothesis.

No configuration shows a statistically significant multi-agent advantage on either dataset. A rule-based baseline is competitive with, or beats, both LLM architectures on AIOps, where the rule-based classifier's 76.0% is roughly 3x every LLM configuration's accuracy—a practical answer for a deploying team: nothing here argues for replacing a working heuristic with an LLM pipeline on this task today. Two reviewer-requested ablations produced small, real, secondary effects: RAG relevance gating reduced a CPU/disk over-retrieval bias (67/75→59/75 guesses mentioning CPU or disk regardless of true cause), modestly improving AIOps accuracy; GPT-4o-mini, in the single-agent role, matched GPT-4o on AIOps (22.7% both) but trailed on RCAEval (26.7% vs. 33.3%) (cost detail in Section IV).

TABLE I
EXACT-MATCH ACCURACY, BOTH DATASETS (N=75 EACH)

Configuration	AIOps 2020		RCAEval RE1	
	Single	Multi	Single	Multi
Baseline	24.0%	17.3%	41.3%	34.7%
Sort-fix	20.0%	18.7%	—	—
Raw-context fix	22.7%	22.7%	33.3%	37.3%
+RAG	24.0%	24.0%	29.3%	34.7%
+RAG, gated	22.7%	25.3%	29.3%	30.7%
Fair test[1]	25.3%	4.0%	—	—
Rule-based[2]	76.0%		36.0%	—

E. Confidence Calibration

Verbalized self-reported confidence does *not* discriminate correct from incorrect hypotheses: mean confidence for incorrect hypotheses is nearly identical to correct ones on both datasets (AIOps, N=75: 0.86 correct vs. 0.85 incorrect; RCAEval: 0.90 vs. 0.87), and at the proposed 0.75 gate, 96–100% of incorrect hypotheses would pass in both—the same rate as correct ones. Self-consistency confidence (5 resamples, temperature 0.7, majority-vote, fixed N=30 AIOps subsample) also fails (0.98 correct vs. 0.93 incorrect): the model is frequently *consistently* wrong—the same error repeats across resamples, a systematic-bias failure resampling cannot fix, unlike sampling-noise miscalibration—a real, publishable safety finding about the limits of LLM confidence gating here, not a gap to paper over.

F. Latency, Cost, and Threshold Sensitivity

On a 15-incident sample, measured (not estimated) wall-clock latency and OpenAI published per-token cost were 0.75s / $0.0013 single-agent vs. 3.53s / $0.0010 multi-agent—the summary-based harness used throughout, not a full iterative tool-calling deployment against live APIs; the two are not directly comparable, and we do not present these as a validated production estimate. Separately, the rule-based baseline's CPU threshold was swept 50–95%: exact-or-partial accuracy ranged 73.3–81.3%, highest at the lowest threshold tried—not strongly sensitive to this choice within a normal range, stated explicitly rather than presenting one threshold as tuned.

G. Falsifier Extension: A Conditional, Mechanistically-Explained Result

None of the six configurations above give multi-agent a capability single-agent could not also have; they only reorganize what Reasoning sees. Flow-of-Action [34] and MA-RCA [35] report large gains from a genuinely new capability instead (executable SOP grounding, or a dedicated Validation Agent testing each hypothesis dynamically). We pre-registered a targeted test of an LLM-based proxy for the latter (Section III-G): add the Falsifier to the raw-context-fix configuration, run once on both datasets, report regardless of direction. Two implementation issues, found in a pre-run

smoke test before any accuracy number was computed, were fixed on implementation grounds, not outcome grounds: free-text alternatives could return a negation ("Low CPU Usage") the keyword scorer misread as a match, fixed by constraining alternatives to the closed vocabulary used throughout; and the initial adversarial prompt refuted nearly every hypothesis, including correct ones, fixed by requiring `confirmed` as the default absent concrete contradicting evidence.

TABLE II
FALSIFIER EXTENSION: EXACT MATCH, SINGLE RUN EACH DATASET

Dataset (N=75)	Single-agent	+Falsifier	McNemar p
AIOps, multi-agent+Falsifier	21.3%	33.3%	0.023
AIOps, single-agent+Falsifier	21.3%	32.0%	0.077
RCAEval (multi-service)	42.7%	24.0%	< 0.001
RCAEval, topology-aware	42.7%	40.0%	0.50

The result is significant in both directions at once. On AIOps (single entity), the extension corrected 11 wrong hypotheses and broke only 2 previously-correct ones—a significant improvement. On RCAEval (a real microservice call graph per incident), it corrected *zero* wrong hypotheses and broke 12—a significant regression. Why: the Falsifier repeatedly overrides a correct root-cause service with a *different* service also anomalous in the same window—textbook symptom propagation, a downstream service degrading because of an upstream cause, not because it is the cause. Lacking service-dependency-graph awareness, the Falsifier cannot tell "anomalous because it is the cause" from "anomalous because it is a victim," impossible on AIOps, where there is only one entity to reason about.

The AIOps result pairs the Falsifier with the multi-agent hypothesis, leaving open whether the gain needs decomposition. A pre-registered control isolates this: the same Falsifier on the (reused) single-agent hypothesis scores 21.3%→32.0%, $p = 0.077$, almost identical to multi-agent (33.3%); a direct incident-level comparison returns $p = 1.0$ (21 right under both, 47 wrong under both, 4 vs. 3 on the rest)—statistically indistinguishable. The Falsifier drives the AIOps gain, not the decomposition it was attached to; crossing $p < 0.05$ in one condition reflects how close the estimates are, not a real architectural difference. The same single-run caveat applies here: the 8-point sampling swing in Section VI.B means this significance reflects within-run pairing, not run-to-run reproducibility.

This is mechanistically explained: validation helps when the alternative space is one entity's own metrics, and hurts when it spans a call graph without topology awareness. We tested the fix directly: give the Falsifier each system's real service dependency graph (verified for Online Boutique and Sock Shop; a naming-convention proxy for Train Ticket's 64 services—so confirmed for two of three systems, Train Ticket's contribution resting on an unverified proxy), instructing it that graph-connected co-anomalies are expected propagation, not independent evidence. This eliminates the regression (24.0%→40.0%, indistinguishable from single-agent's 42.7%, $p = 0.50$ vs. $p < 0.001$ before) but adds no gain: the topology-aware Falsifier made zero overrides across all 75 incidents, trading overconfidence for near-total caution rather than discrimination—harm fixed, value not yet added.

VI. DISCUSSION

A. Design Principles

Three principles motivated the design; we revisit each against Section V. *Context hygiene via decomposition*: the fair context-pollution test shows the clearest result, in the opposite direction: blind, values-withheld metric-name selection defaults to generic, dataset-typical names regardless of which signal is actually anomalous, so Telemetry frequently investigates the wrong evidence before Reasoning ever sees it—the actual finding, not the original unqualified claim. *Asymmetric safety calibration*: an incorrect autonomous action extends an incident while an unnecessary escalation costs minutes, arguing for conservative thresholds under SRE risk-budgeting [30], independent of decomposition's benefit. *Observability*: full logging of every tool call, inference, and state transition is what made Section V-E's confidence-discrimination failure discoverable at all. Section V-G sharpens this further: the capability helping single-entity incidents did not need the decomposition it was attached to ($p = 1.0$)—value comes from the capability, not agent count (Section II-C).

B. Limitations and Threats to Validity

The evaluation has several limitations. AIOps 2020 and RCAEval RE1 each cover a specific set of systems and fault types that may not represent all cloud environments; RCAEval's ground truth is an injected fault label, not a full causal narrative, capping how finely "exact match" is defined, and AIOps's annotations have an analogous limitation. Confidence thresholds need recalibration against live data—Section V-E shows no threshold in the practical range currently discriminates safely. All configurations ran once: re-running an *unchanged* prompt moved its own score by 8 points from sampling noise alone (RCAEval single-agent, 41.3%→33.3%), larger than most reported effects; a repeated-run protocol was not performed, so point estimates are indicative. On validity: the single-agent baseline uses the same tool schema and prompt length, but both prompts were written by the same author, a potential bias either way (*internal*); results come from two public benchmarks, so generalization elsewhere is plausible but unconfirmed (*external*); and "partial match" conflates cases of varying operator usefulness, so the headline comparison uses exact match, the stricter metric (*construct*).

C. Security and Deployment Considerations

Autonomous LLM agents executing production remediations [20] introduce a distinct security surface: prompt injection via telemetry (mitigate by sanitizing tool-boundary inputs, treating outputs as data, not instructions), confidence manipulation via a poisoned RAG index (easier given Section V-E's weak confidence-correctness link; mitigate via restricted write access and ingestion audits), and audit-log integrity (`InvestigationState` should be append-only,

integrity-verified)—none disqualifies autonomous operation, but security review is a prerequisite. Deployment raises further concerns: alert-burst queuing (e.g., Alertmanager grouping); the 0.75 Tier 1 threshold should not be adopted without re-running Section V-E's discrimination analysis against a deployment's own history; GPT-4o updates may shift calibration without notice, arguing for pinned versions or an open model (Llama 3 [28]); and the RAG index needs active maintenance, since recency decay ($\lambda = 0.95$/month) only partially offsets staleness.

D. Future Work

Motivated by Section V-G: verify Train Ticket's full call graph beyond the naming-convention proxy used here, and sharpen the override decision. Beyond that: a production evaluation with real incident data and fault-injection studies [31]; a repeated-run protocol isolating real effects from sampling noise; fine-tuning a smaller open model (Llama 3 [28]) and re-measuring single- vs. multi-agent; and a public benchmark on AIOps 2020/RCAEval against Flow-of-Action [34] and MA-RCA [35].

VII. Conclusion

We presented NetOpsAgent, a hierarchical multi-agent LLM framework for autonomous cloud incident management, decomposing investigation into four agents coordinated via a shared state object. On two independent real fault-injection benchmarks (AIOps Challenge 2020, RCAEval RE1, $N = 75$ each) across six configurations, decomposition did not significantly improve on a single-agent baseline, including under a configuration designed specifically to test the architecture's own context-hygiene hypothesis—an honest, triangulated finding that multi-agent LLM decomposition's benefit for infrastructure triage is more setup-dependent than prior work (Flow-of-Action [34], MA-RCA [35]) has reported. We also found, and report in full, that neither verbalized nor self-consistency-based confidence discriminates correct from incorrect hypotheses here, materially limiting the safety guarantee our confidence-gated autonomy mechanism can currently provide.

Finally, a targeted Falsifier extension adversarially validating the Reasoning agent's hypothesis—mirroring the validation mechanism recent multi-agent RCA systems report gains from—produced a real, significant effect in both directions: improving accuracy on single-entity incidents (AIOps), degrading it on multi-service incidents (RCAEval), because without service-dependency-graph awareness it mistakes propagated symptoms for root causes once an incident spans more than one service. A service-topology fix (Section V-G) removes the RCAEval regression but does not yet turn it into a gain—honest, not rounded up. A control test shows the AIOps gain does not require decomposition: a single agent given the same Falsifier scores indistinguishably from multi-agent ($p = 1.0$). This paper's central claim is therefore not that multi-agent architecture adds value for LLM-based RCA, but that a specific *capability*—adversarial verification—does, independent of agent count. Practically: skip decomposition, invest in adversarial verification with topology awareness, and confirm an LLM pipeline earns its complexity over a heuristic. All findings are reported because a verification or autonomy mechanism is only useful if its failure modes are visible; the public implementation and harness make all of them, and any future re-measurement, reproducible.

Acknowledgment

The author thanks the LangGraph and Google ADK communities.

References

[1] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly, 2016.
[2] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
[3] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
[4] LangChain Team, "LangGraph: Building stateful, multi-actor applications with LLMs," 2024.
[5] J. Zhao et al., "AIOps Challenge 2020: KPI anomaly detection," 2020.
[6] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.
[7] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
[8] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," arXiv:2308.00352, 2023.
[9] S. Patil et al., "Gorilla: Large language model connected with massive APIs," arXiv:2305.15334, 2023.
[10] C. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
[11] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019.
[12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, 2021.
[13] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
[14] C. Lim, N. R. Murphy, and B. Beyer, "On-call and incident response," in *The Site Reliability Workbook*. O'Reilly, 2018.
[15] M. Bjorklund, "NETCONF Configuration Protocol," RFC 6241, IETF, 2011.
[16] B. Rabenstein and J. Volz, "Prometheus: A next-generation monitoring system," in *Proc. USENIX LISA*, 2015.
[17] H. Liu et al., "LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis," arXiv:2308.07610, 2023.
[18] T. Winters, T. Manshreck, and H. Wright, *Software Engineering at Google*. O'Reilly, 2020.
[19] C. Jimenez et al., "SWE-bench verified," arXiv:2405.15793, 2024.
[20] B. Burns et al., "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, 2016.
[21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, 1947.
[22] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, 1960.
[23] Y. Guo et al., "Exploring the feasibility of ChatGPT for event extraction," arXiv:2303.03836, 2023.
[24] X. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proc. EuroSys*, 2024.
[25] W. Luo et al., "An unnamed knowledge graph approach to root-cause analysis for microservice systems," in *Proc. ISSRE*, 2022.
[26] D. Roy et al., "Exploring LLM-based agents for root cause analysis," arXiv:2403.04123, 2024.
[27] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.
[28] Meta AI, "Llama 3 model card," 2024.
[29] Elastic, "Elasticsearch: The official distributed search and analytics engine," 2024.
[30] N. R. Murphy et al., "Embracing risk," in *Site Reliability Engineering*. O'Reilly, 2016, ch. 3.
[31] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
[32] OpenTelemetry Authors, "OpenTelemetry specification," 2024.
[33] C. Guo et al., "On calibration of modern neural networks," in *Proc. ICML*, 2017.
[34] C. Pei et al., "Flow-of-Action: SOP enhanced LLM-based multi-agent system for root cause analysis," in *Companion Proc. ACM Web Conference (WWW Companion '25)*, 2025.
[35] F. Fu et al., "Leveraging multi-agent framework for root cause analysis," *Complex & Intelligent Systems*, vol. 12, no. 1, art. 4, 2026.
[36] J. Xu et al., "Rethinking the value of multi-agent workflow: a strong single agent baseline," arXiv:2601.12307, 2026.
[37] J. Su and C. Wu, "When do multi-agent systems outperform? Analysing the learning efficiency of agentic systems," arXiv:2602.08272, 2026.